

You can initiate the scan process by calling [StartSoftwareTriggerAsync](#).

Depending on the value of [IsDisabledOnDataReceived](#) the scanner might scan only one barcode then stop or scan continuously until you call [StopSoftwareTriggerAsync](#).

Set the desired value of [IsDisabledOnDataReceived](#) to control the scanner behavior when a barcode is decoded.

 Expand table

Value	Description
True	Scan only one barcode then stop
False	Continuously scan barcodes without stopping

See also

- [JustScanIt - Windows Store app](#)
- [BarcodeScanner sample](#)

Feedback

Was this page helpful? [Yes](#) [No](#)

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

Host a camera barcode scanner preview in a UWP application

Article • 05/04/2023

Requires Windows 10 Version 1803 or later.

This topic describes how to host a camera barcode scanner preview in a UWP application.

Step 1: Setup your camera preview

See [Display the camera preview](#) for instructions on how to quickly display the camera preview stream within a XAML page in a Universal Windows Platform (UWP) app. When complete, return to this topic for camera barcode scanner specific modifications.

Step 2: Edit the capability declarations in your app manifest

Edit the capability declarations in the app manifest to prevent users from receiving the microphone consent prompt.

1. In Microsoft Visual Studio, in **Solution Explorer**, open the designer for the application manifest by double-clicking the **package.appxmanifest** item.
2. Select the **Capabilities** tab.
3. Uncheck the box for **Microphone**.

Step 3: Add a `using` directive to support media capture

Csharp

```
using Windows.Media.Capture;
```

Step 4: Set up your media capture initialization settings

The following snippet shows how to initialize a [MediaCaptureInitializationSettings](#) object with the following settings:

- [BarcodeScanner.VideoDeviceId](#)
- [StreamingCaptureMode.Video](#)
- [PhotoCaptureSource.VideoPreview](#)

Csharp

```
private void InitCaptureSettings()
{
    _captureInitSettings = new MediaCaptureInitializationSettings();
    _captureInitSettings.VideoDeviceId = BarcodeScanner.VideoDeviceId;
    _captureInitSettings.StreamingCaptureMode = StreamingCaptureMode.Video;
    _captureInitSettings.PhotoCaptureSource =
    PhotoCaptureSource.VideoPreview;
}
```

Step 5: Associate the MediaCapture object with a camera barcode scanner

Replace the existing [InitializeAsync](#) method of the [MediaCapture](#) object in `StartPreviewAsync()` (see [Step 1: Setup your camera preview](#)) with the following:

Csharp

```
try
{
    mediaCapture = new MediaCapture();
    await mediaCapture.InitializeAsync(InitCaptureSettings());

    displayRequest.RequestActive();
    DisplayInformation.AutoRotationPreferences =
    DisplayOrientations.Landscape;
}
```

Tip

See [Display the camera preview](#) for more advanced topics on hosting a camera preview in your UWP application.

See also

- [JustScanIt - Windows Store app](#) 
- [BarcodeScanner sample](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Enable or disable the software decoder for the camera barcode scanner

Article • 05/06/2023

This topic explains how to enable or disable the software decoder for the camera barcode scanner.

The software decoder can be disabled if you do not want to use camera barcode scanner or if you have acquired a 3rd party decoder that works with the [BarcodeScanner](#) APIs.

In Windows 10 Version 1803 or later, the software decoder is installed and enabled by default.

⚠ Note

The software decoder built into Windows 10/11 is provided by [Digimarc Corporation](#) [↗].

Enable or disable using the system registry

The software decoder that ships with Windows can be enabled or disabled through the system registry. Just add the registry key *InboxDecoder* under *HKLM\Software\Microsoft\PointOfService\BarcodeScanner* and set the *Enable* value as described below.

[↗](#) Expand table

Value name	Value Type	Value
Enable	DWORD	1 (default) - Enables the software decoder that ships with Windows 0 - Disables the software decoder that ships with Windows

Here is an example Registration Entries (.reg) file that disables the software decoder that ships with Windows:

text

```
Windows Registry Editor Version 5.00
```

```
[HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\PointOfService\BarcodeScanner\InboxDe
```

```
coder]
"Enable"=dword:0000000
```

Here is an example Registration Entries (.reg) file that enables the software decoder that ships with Windows:

text

Windows Registry Editor Version 5.00

```
[HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\PointOfService\BarcodeScanner\InboxDecoder]
"Enable"=dword:0000001
```

⊗ Caution

Serious problems might occur if you modify the registry incorrectly. For added protection, back up the registry before you modify it. Then, you can restore the registry if a problem occurs. For more information about how to back up and restore the registry, click the following article number to view the article in the Microsoft Knowledge Base:

[322756](#)  How to back up and restore the registry in Windows.

See also

- [JustScanIt - Windows Store app](#) 
- [BarcodeScanner sample](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Camera barcode scanner symbologies

Article • 05/04/2023

Requires Windows 10 Version 1803 or later.

A camera barcode scanner consists of a standard lens camera with auto-focus capabilities (attached to a computer) combined with a software decoder. This topic describes the symbologies supported by the software decoder.

ⓘ **Note**

The software decoder built into Windows 10/11 is provided by [Digimarc Corporation](#) [↗].

To determine the specific symbologies supported by a barcode scanner, call [GetSupportedSymbologiesAsync](#).

Additional symbologies might be supported by dedicated barcode scanner devices that have built-in hardware decoders, please contact your barcode scanner manufacturer for details.

1D Symbologies

[⌵](#) Expand table

Symbology	Min Build	Sample
Code 39	17134	

Symbology	Min Build	Sample
Code 128	17134	 1234567890
Databar Omnidirectional	17134	 (01)12345678901231
Databar Stacked	17134	
EAN-8	17134	 1234 5670

Symbology	Min Build	Sample
EAN-13	17134	
Interleaved 2 of 5	17134	
UPC-A	17134	
UPC-E	17134	

2D Symbologies

[Expand table](#)

Symbology	Min Build	Sample
PDF417	19044	
QR Code	17134	

Digital Watermark

[Expand table](#)

Symbology	Min Build	Sample
GS1-DWCode	17134	Scan the image of a package below with your camera barcode scanner application to see GS1DWCode in action. The image is encoded with UPCA 856107006854. Please visit http://www.digimarc.com for more information about GS1DWCode capabilities.



See also

- [JustScanIt - Windows Store app](#) ↗
- [BarcodeScanner sample](#) ↗

Feedback

Was this page helpful?

[Provide product feedback](#) ↗ | [Get help at Microsoft Q&A](#)

Barcode Scanner Advanced Configuration

Article • 01/09/2024

⊗ **Caution**

These defaults are set after extensive testing to optimize a balance between decode performance and CPU utilization while not sacrificing battery life. Changing these values can not only increase or decrease scanning performance, but also impact overall system performance by consuming more CPU cycles and decrease battery life. Use caution and test the impact of each setting changed thoroughly.

The settings which control the behavior of the software barcode decoder are stored in the system registry under `HKLM\Software\Microsoft\PointOfService\InboxDecoder`. If you do not see any settings written to the registry in this location, it means they are following the default settings defined here.

Enable / Disable

The software decoder can be disabled to prevent enumeration as a virtual barcode scanner using this registry setting. Disabling the software decoder will have no affect on the use of the camera for other purposes. Be aware that applications may make assumptions that the decoder is always available, so be sure to test your scenarios that rely on camera access fully if you do disable the decoder.

 Expand table

Value name	Value Type	Value	Notes
Enable	DWord	1	Enabled
		0	Disabled

MinimumQuietTimeMilliseconds

MinimumQuietTimeMilliseconds defines a timeframe where the camera barcode decoder must not see a barcode after a successful read. This is to prevent an attempt to acquire a single barcode from returning multiple results for the same scannable item. Moving the barcode out of view and back into view will successfully read the barcode

again. Lowering this value could result in the decoder to return multiple results for a single scannable item.

[Expand table](#)

Value name	Value Type	Value	Notes
MinimumQuietTimeMilliseconds	DWord	1200	Default ms

PreferredVideoFrameRate

PreferredVideoFrameRate default is set at 5 frames per second to balance performance with CPU utilization and battery consumption. Increasing the number of frames per second can consume significantly more CPU cycles and battery consumption. Reducing the value below 5 frames per second can impact scanning performance.

[Expand table](#)

Value name	Value Type	Value	Notes
PreferredVideoFrameRate	DWord	1	Minimum fps
		30	Maximum fps
		5	Default fps

PreferredVideoFrameWidth

PreferredVideoFrameWidth by default is set to 1920 pixels. Increasing the pixel depth can require more CPU cycles and battery consumption to decode frames scanned. Decreasing the pixel depth can reduce the CPU cycles, however it will also reduce the detection rate.

[Expand table](#)

Value name	Value Type	Value	Notes
PreferredVideoFrameWidth	DWord	800	Minimum px
		2048	Maximum px
		1920	Default px

PreferredVideoFrameHeight

PreferredVideoFrameHeight default is set to 0, which is interpreted as the smallest height paired with the PreferredVideoFrameWidth. Similar to PreferredVideoFrameWidth, setting the frame height to a large height can increase CPU utilization and battery consumption, with very little benefit.

 Expand table

Value name	Value Type	Value	Notes
PreferredVideoFrameHeight	DWord	0	Default: <i>Smallest height paired with width</i>

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Magnetic stripe reader

Article • 05/06/2023

A magnetic stripe reader can scan certain types of cards that have a magnetic stripe on them, such as credit cards, driver's licenses, and store loyalty cards.

Using a magnetic stripe reader in your Universal Windows Platform (UWP) application is similar to using any other point of service device, in that you have to enumerate the available devices, create the device object, and claim the device before you can use it. Follow the documentation under [Getting started with Point of Service](#) for more information.

In this section

Topic	Description
Obtain and understand magnetic stripe data	Learn how to obtain and interpret the data from a magnetic stripe.

Obtain and understand magnetic stripe data

Article • 05/06/2023

This topic describes how to obtain and interpret the data from a magnetic stripe reader using Universal Windows Platform (UWP) Point of Service (POS) APIs.

Subscribe to *DataReceived events

Once you've set up your magnetic stripe reader in your application using the steps outlined in [Getting started with Point of Service](#), you're ready to start getting data from it.

Whenever the reader recognizes a swiped card, it will raise one of three events:

- [AamvaCardDataReceived Event](#): Occurs when a motor vehicle card is swiped.
- [BankCardDataReceived Event](#): Occurs when a bank card is swiped.
- [VendorSpecificDataReceived Event](#): Occurs when a vendor-specific card is swiped.

Your application only needs to subscribe to the events that are supported by the magnetic stripe reader. You can see what types of cards are supported with [MagneticStripeReader.SupportedCardTypes](#).

The following code demonstrates subscribing to the three *DataReceived events:

C#

```
private void SubscribeToEvents(ClaimedMagneticStripeReader claimedReader,
MagneticStripeReader reader)
{
    foreach (var type in reader.SupportedCardTypes)
    {
        if (item == MagneticStripeReaderCardTypes.Aamva)
        {
            claimedReader.AamvaCardDataReceived +=
Reader_AamvaCardDataReceived;
        }
        else if (item == MagneticStripeReaderCardTypes.Bank)
        {
            claimedReader.BankCardDataReceived +=
Reader_BankCardDataReceived;
        }
        else if (item == MagneticStripeReaderCardTypes.ExtendedBase)
        {
            claimedReader.VendorSpecificDataReceived +=
```

```
Reader_VendorSpecificDataReceived;  
    }  
}  
}
```

The event handler will be passed the [ClaimedMagneticStripeReader](#) and an *args* object, whose type will vary depending on the event:

- **AamvaCardDataReceived** event:
[MagneticStripeReaderAamvaCardDataReceivedEventArgs Class](#)
- **BankCardDataReceived** event:
[MagneticStripeReaderBankCardDataReceivedEventArgs Class](#)
- **VendorSpecificDataReceived** event:
[MagneticStripeReaderVendorSpecificCardDataReceivedEventArgs Class](#)

Get the data

For the **AamvaCardDataReceived** and **BankCardDataReceived** events, you can get some of the data directly from the *args* object. The following example demonstrates getting a few properties and assigning them to member variables:

C#

```
private string _accountNumber;  
private string _expirationDate;  
private string _firstName;  
  
private void Reader_BankCardDataReceived(  
    ClaimedMagneticStripeReader sender,  
    MagneticStripeReaderBankCardDataReceivedEventArgs args)  
{  
    _accountNumber = args.AccountNumber;  
    _expirationDate = args.ExpirationDate;  
    _firstName = args.FirstName;  
    // etc...  
}
```

However, some data, including all data from the **VendorSpecificDataReceived** event, must be retrieved through the **Report** object, which is a property of the *args* parameter. This is of type [MagneticStripeReaderReport](#).

You can use the [CardType](#) property to figure out what type of card has been swiped, and then use that to inform how you interpret the data from [Track1](#), [Track2](#), [Track3](#), and [Track4](#).

The data from each of the tracks are represented as [MagneticStripeReaderTrackData](#) objects. From this class, you can get the following types of data:

- [Data](#): The raw or decoded data.
- [DiscretionaryData](#): The discretionary data.
- [EncryptedData](#): The encrypted data.

The following code snippet gets the report and the track data, and then checks the card type:

C#

```
private void GetTrackData(MagneticStripeReaderBankCardDataReceivedEventArgs args)
{
    MagneticStripeReaderReport report = args.Report;
    IBuffer track1 = report.Track1.Data;
    IBuffer track2 = report.Track2.Data;
    IBuffer track3 = report.Track3.Data;
    IBuffer track4 = report.Track4.Data;

    if (report.CardType == MagneticStripeReaderCardTypes.Aamva)
    {
        // Card type is AAMVA
    }
    else if (report.CardType == MagneticStripeReaderCardTypes.Bank)
    {
        // Card type is bank card
    }
    else if (report.CardType == MagneticStripeReaderCardTypes.ExtendedBase)
    {
        // Card type is vendor-specific
    }
    else if (report.CardType == MagneticStripeReaderCardTypes.Unknown)
    {
        // Card type is unknown
    }
}
```

Support and feedback

Find answers to your questions

Have questions? Ask us on either our [Docs Q&A forum with the UWP tag](#) or on [Stack Overflow with the pointofservice tag](#).

Help us locate your questions:

- Add the [pointofservice](#) tag to your question on Stack Overflow.
- Include the "UWP" term in your post on the Q&A forum

See also

- [Magnetic stripe reader](#)
- [ClaimedMagneticStripeReader Class](#)
- [MagneticStripeReaderAamvaCardDataReceivedEventArgs Class](#)
- [MagneticStripeReaderBankCardDataReceivedEventArgs Class](#)
- [MagneticStripeReaderVendorSpecificCardDataReceivedEventArgs Class](#)
- [MagneticStripeReaderReport](#)
- [MagneticStripeReaderTrackData](#)

POSPrinter

Article • 05/06/2023

This section provides guidance for creating Universal Windows Platform (UWP) apps that use a Point of Service (POS) printer.

In this section

Topic	Description
Epson ESC/POS with formatting	Learn how to use the ESC/POS command language to format text, such as bold and double size characters, for your Point of Service printer.

Epson ESC/POS with formatting

Article • 05/06/2023

Learn how to use the ESC/POS command language to format text, such as bold and double size characters, for your Point of Service printer.

Important APIs

- [PointofService Printer](#)
- [Windows.Devices.PointOfService](#)

ESC/POS usage

Windows Point of Service supports a variety of printers, including several Epson TM series printers (for a full list of supported printers, see the [PointofService Printer](#) page). Windows supports printing through the ESC/POS printer control language, which provides efficient and functional commands for communicating with your printer.

ESC/POS is a command system created by Epson used across a wide range of POS printer systems, aimed at avoiding incompatible command sets by providing universal applicability. Most modern printers support ESC/POS.

All commands start with the ESC character (ASCII 27, HEX 1B) or GS (ASCII 29, HEX 1D), followed by another character that specifies the command. Normal text is simply sent to the printer, separated by line breaks.

The [Windows PointOfService API](#) provides much of that functionality for you via the `Print()` or `PrintLine()` methods. However, to get certain formatting or to send specific commands, you must use ESC/POS commands, built as a string and sent to the printer.

Example using bold and double size characters

The example below shows how to use ESC/POS commands to print in bold and double sized characters. Note that each command is built as a string, then inserted into the `printJob` calls.

C#

```
// ... prior plumbing code removed for brevity
// this code assumed you've already created a receipt print job (printJob)
// and also that you've already checked the PosPrinter Capabilities to
// verify that the printer supports Bold and DoubleHighDoubleWide print
```

modes

```
const string ESC = "\u001B";
const string GS = "\u001D";
const string InitializePrinter = ESC + "@";
const string BoldOn = ESC + "E" + "\u0001";
const string BoldOff = ESC + "E" + "\0";
const string DoubleOn = GS + "!" + "\u0011"; // 2x sized text (double-high
+ double-wide)
const string DoubleOff = GS + "!" + "\0";

printJob.Print(InitializePrinter);
printJob.PrintLine("Here is some normal text.");
printJob.PrintLine(BoldOn + "Here is some bold text." + BoldOff);
printJob.PrintLine(DoubleOn + "Here is some large text." + DoubleOff);

printJob.ExecuteAsync();
```

For more information on ESC/POS, including available commands, see the [ESC/POS Command Reference Revision 2.60](#) [↗](#). For details on [Windows.Devices.PointOfService](#) and all the available functionality, see [PointofService Printer](#) on MSDN.

Sensors

Article • 05/06/2023

Sensors let your app know the relationship between a device and the physical world around it. Sensors can tell your app the direction, orientation, and movement of the device. These sensors can help make your game, augmented reality app, or utility app more useful and interactive by providing a unique form of input, such as using the motion of the device to arrange the characters on the screen or to simulate being in a cockpit and using the device as the steering wheel.

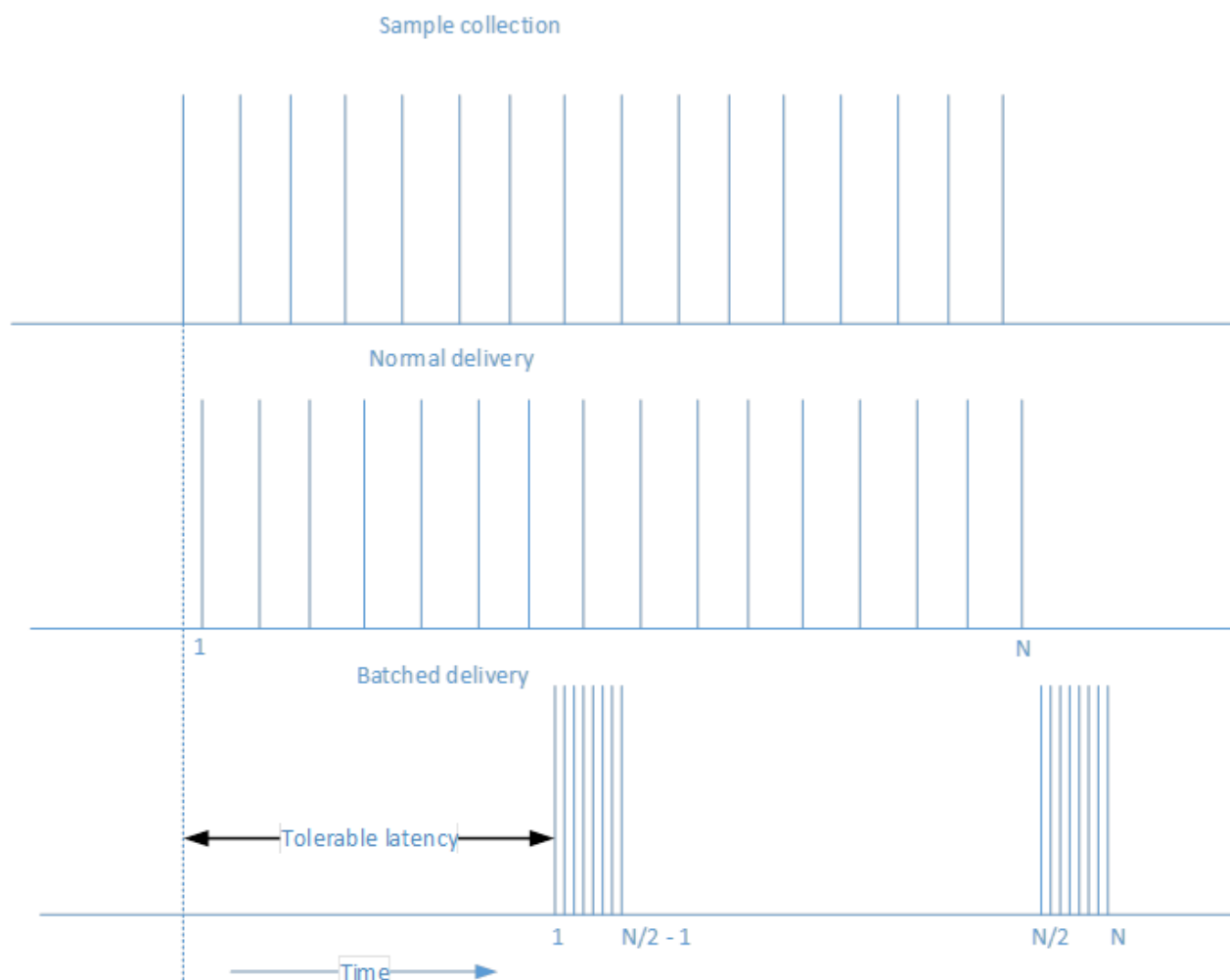
As a general rule, decide from the outset whether your app will depend exclusively on sensors or if sensors will just offer an additional control mechanism. For example, a driving game using a device as a virtual steering wheel could alternatively be controlled through an on-screen GUI – this way, the app works regardless of the sensors available on the system. On the other hand, a marble tilt maze could be coded to only work on systems that have the appropriate sensors. You must make the strategic choice of whether to fully rely on sensors. Note that a mouse/touch control scheme trades immersion for greater control.

Topic	Description
Calibrate sensors	Sensors in a device based on the magnetometer – the compass, inclinometer and orientation sensor - can become in need of calibration due to environmental factors. The MagnetometerAccuracy enumeration can help determine a course of action when your device is in need of calibration.
Sensor orientation	Sensor data from the OrientationSensor classes is defined by their reference axes. These axes are defined by the device's landscape orientation and rotate with the device as the user turns it.
Use the accelerometer	Learn how to use the accelerometer to respond to user movement.
Use the compass	Learn how to use the compass to determine the current heading.
Use the gyrometer	Learn how to use the gyrometer to detect changes in user movement.
Use the inclinometer	Learn how to use the inclinometer to determine pitch, roll, and yaw.
Use the light sensor	Learn how to use the ambient light sensor to detect changes in lighting.

Topic	Description
Use the orientation sensor	Learn how to use the orientation sensors to determine the device orientation.

Sensor batching

Some sensors support the concept of batching. This will vary depending on the individual sensor available. When a sensor implements batching, it collects several points of data over a specified time interval and then transfers all of that data at one time. This is different from normal behavior where a sensor reports its findings as soon as it performs a reading. Consider the following diagram which shows how data is collected and then delivered, first with normal delivery and then with batched delivery.



The primary advantage for sensor batching is prolonging battery life. When the data is not sent immediately, that saves on processor power and prevents the data from needing to be immediately processed. Parts of the system can sleep until they are needed, which generates a significant power savings.

You can influence how often the sensor sends batches by adjusting the latency. For example, the [Accelerometer](#) sensor has the [ReportLatency](#) property. When this property is set for an application, the sensor will send data after the specified amount of time. You can control how much data is accumulated over a given latency by setting the [ReportInterval](#) property.

There are a couple of caveats to keep in mind with respect to setting the latency. The first caveat is that each sensor has a [MaxBatchSize](#) that it can support based on the sensor itself. This is the number of events that the sensor can cache before it is forced to send them. If you multiply [MaxBatchSize](#) by [ReportInterval](#), that determines the maximum [ReportLatency](#) value. If you specify a higher value than this, the maximum latency will be used so that you do not lose data. In addition, multiple applications can each set a desired latency. In order to meet the needs of all applications, the shortest latency period will be used. Because of these facts, the latency you set in your application may not match the observed latency.

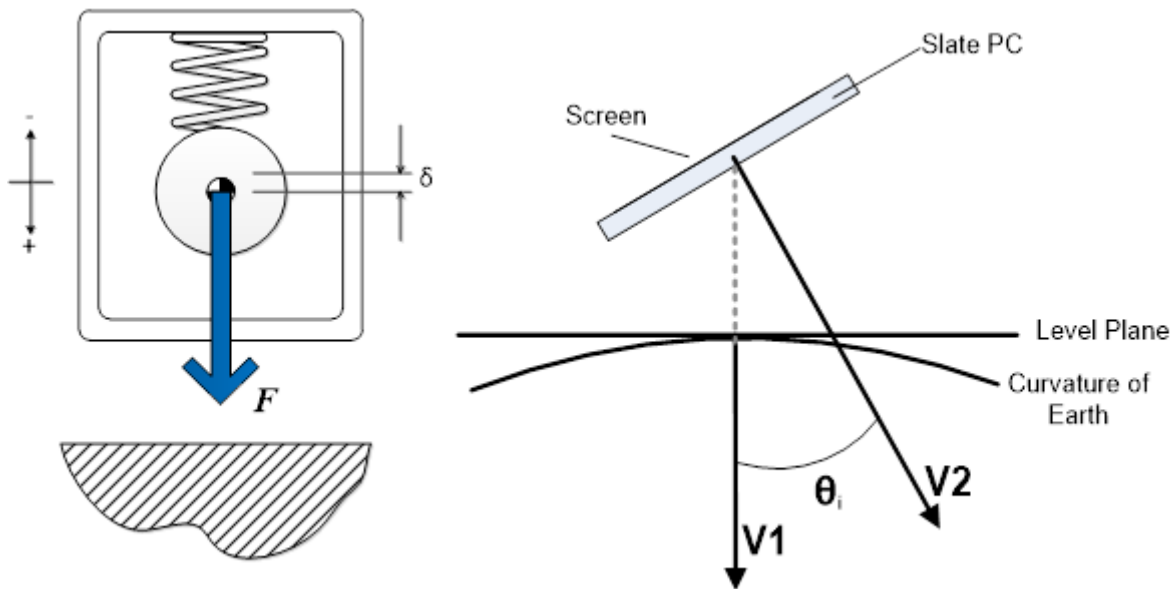
If a sensor is using batch reporting, calling [GetCurrentReading](#) will clear the current batch of data and start a new latency period.

Accelerometer

The [Accelerometer](#) sensor measures G-force values along the X, Y, and Z axes of the device and is great for simple motion-based applications. Note that G-force values include acceleration due to gravity. If the device has the [SimpleOrientation](#) of [FaceUp](#) on a table, then the accelerometer would read -1 G on the Z axis. Thus, accelerometers do not necessarily measure just coordinate acceleration – the rate of change of velocity. When using an accelerometer, make sure to differentiate between the gravitational vector from gravity and the linear acceleration vector from motion. Note that the gravitational vector should normalize to 1 for a stationary device.

The following diagrams illustrate:

- V1 = Vector 1 = Force due to gravity
- V2 = Vector 2 = -Z axis of device chassis (points out of back of screen)
- Θ_i = Tilt angle (inclination) = angle between -Z axis of device chassis and gravity vector



Apps that might use the accelerometer sensor include a game where a marble on the screen rolls in the direction you tilt the device (gravitational vector). This type of functionality closely mirrors that of the [Inclinometer](#) and could also be done with that sensor by using a combination of pitch and roll. Using the accelerometer's gravity vector simplifies this somewhat by providing an easily mathematically manipulated vector for device tilt. Another example would be an app that makes a whip's cracking sound when the user flicks the device through the air (linear acceleration vector).

For an example implementation, see the [accelerometer sample](#) [↗](#).

Activity sensor

The [Activity](#) sensor determines the current status of the device attached to the sensor. This sensor is frequently used in fitness applications to keep track of when a user carrying a device is running or walking. See [ActivityType](#) for a list of possible activities that can be detected by this sensor API.

For an example implementation, see the [activity sensor sample](#) [↗](#).

Altimeter

The [Altimeter](#) sensor returns a value that indicates the altitude of the sensor. This enables you to keep track of a change in altitude in terms of meters from sea level. One example of an app that might use this would be a running app that keeps track of the elevation changes during a run to calculate the calories burned. In this case, this sensor data could be combined with the [Activity](#) sensor to provide more accurate tracking information.

For an example implementation, see the [altimeter sample](#) [↗].

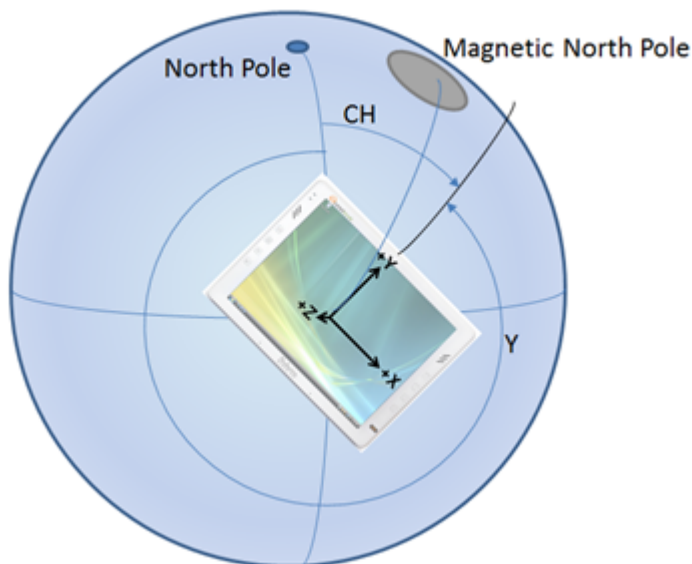
Barometer

The **Barometer** sensor enables an application to get barometric readings. A weather application could use this information to provide the current atmospheric pressure. This could be used to provide more detailed information and predict potential weather changes.

For an example implementation, see the [barometer sample](#) [↗].

Compass

The **Compass** sensor returns a 2D heading with respect to magnetic north based on the horizontal plane of the earth. The compass sensor should not be used in determining specific device orientation or for representing anything in 3D space. Geographical features can cause natural declination in the heading, so some systems support both **HeadingMagneticNorth** and **HeadingTrueNorth**. Think about which one your app prefers, but remember that not all systems will report a true north value. The gyrometer and magnetometer (a device measuring magnetic strength magnitude) sensors combine their data to produce the compass heading, which has the net effect of stabilizing the data (magnetic field strength is very unstable due to electrical system components).

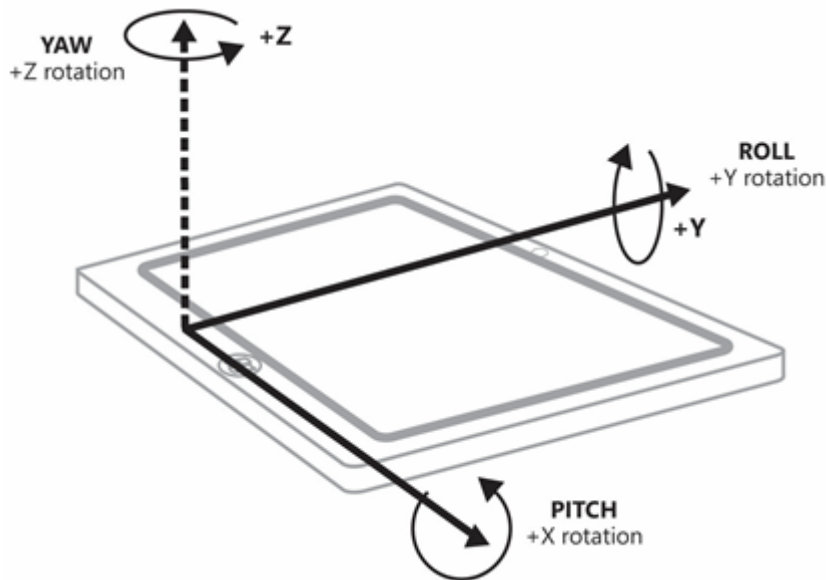


Apps that want to display a compass rose or navigate a map would typically use the compass sensor.

For an example implementation, see the [compass sample](#) [↗].

Gyrometer

The **Gyrometer** sensor measures angular velocities along the X, Y, and Z axes. These are very useful in simple motion-based apps that do not concern themselves with device orientation but care about the device rotating at different speeds. Gyrometers can suffer from noise in the data or a constant bias along one or more of the axes. You should query the accelerometer to verify whether the device is moving in order to determine if the gyrometer suffers from a bias, and then compensate accordingly in your app.

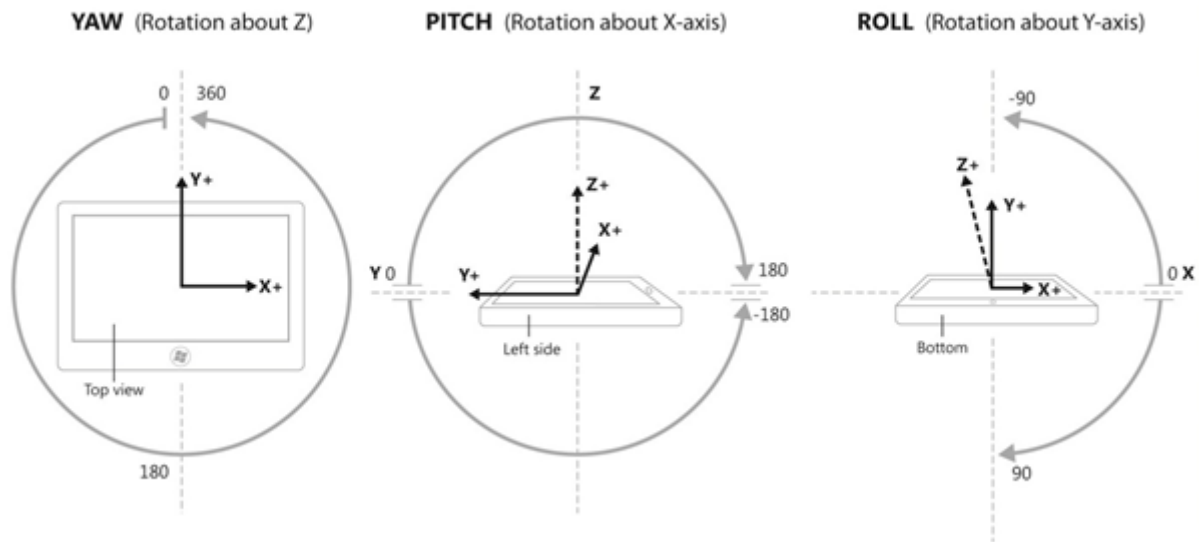


An example of an app that could use the gyrometer sensor is a game that spins a roulette wheel based on a quick rotational jerk of the device.

For an example implementation, see the [gyrometer sample](#).

Inclinometer

The **Inclinometer** sensor specifies the yaw, pitch, and roll values of a device and work best with apps that care about how the device is situated in space. Pitch and roll are derived by taking the accelerometer's gravity vector and by integrating the data from the gyrometer. Yaw is established from magnetometer and gyrometer (similar to compass heading) data. Inclinometers offer advanced orientation data in an easily digestible and understandable way. Use inclinometers when you need device orientation but do not need to manipulate the sensor data.



Apps that change their view to match the orientation of the device can use the inclinometer sensor. Also, an app that displays an airplane that matches the yaw, pitch, and roll of the device would also use the inclinometer readings.

For an example implementation, see the inclinometer sample

<https://github.com/Microsoft/Windows-universal-samples/tree/main/Samples/Inclinometer>.

Light sensor

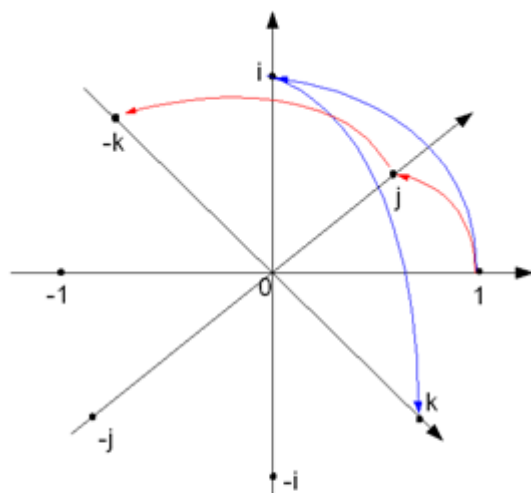
The **Light** sensor is capable of determining the ambient light surrounding the sensor. This enables an app to determine when the light setting surrounding a device has changed. For example, a user with a slate device might walk from indoors to outdoors on a sunny day. A smart application could use this value to increase the contrast between the background and the font being rendered. That would make the content still readable in the brighter, outdoor setting.

For an example implementation, see the [light sensor sample](#).

Orientation sensor

Device orientation is expressed through both quaternion and a rotation matrix. The **OrientationSensor** offers a high degree of precision in determining how the device is situated in space with respect to absolute heading. The **OrientationSensor** data is derived from the accelerometer, gyrometer, and magnetometer. As such, both the inclinometer and compass sensors can be derived from the quaternion values. Quaternions and rotation matrices lend themselves well to advanced mathematical manipulation and are often used in graphical programming. Apps using complex

manipulation should favor the orientation sensor as many transforms are based off of quaternions and rotation matrices.



Graphical representation of quaternion units product as 90°-rotation in 4D-space

$$\begin{aligned} ij &= k \\ ji &= -k \\ ij &= -ji \end{aligned}$$

The orientation sensor is often used in advanced augmented reality apps that paint an overlay on your surroundings based on the direction the back of the device is pointing.

For an example implementation, see the [orientation sensor sample](#) ↗.

Pedometer

The **Pedometer** sensor keeps track of the number of steps taken by the user carrying the connected device. The sensor is configured to keep track of the number of steps over a given time period. Several fitness applications like to keep track of the number of steps taken in order to help the user set and reach various goals. This information can then be collected and stored to show progress over time.

For an example implementation, see the [pedometer sample](#) ↗.

Proximity sensor

The **Proximity** sensor can be used to indicate whether or not objects are detected by the sensor. In addition to determining whether or not an object is within range of the device, the proximity sensor also can determine the distance to the detected object. One example where this could be used is with an application that wants to emerge from a sleep state when a user comes within a specified range. The device could be in a low-

powered sleep state until the proximity sensor detects an object, and then could enter a more active state.

For an example implementation, see the [proximity sensor sample](#).

Simple orientation

The [SimpleOrientationSensor](#) detects the current quadrant orientation of the specified device or it's face-up or face-down. It has six possible [SimpleOrientation](#) states (**NotRotated**, **Rotated90**, **Rotated180**, **Rotated270**, **FaceUp**, **FaceDown**).

A reader app that changes its display based on the device being held parallel or perpendicular to the ground would use the values from the SimpleOrientationSensor to determine how the device is being held.

For an example implementation, see the [simple orientation sensor sample](#).

Calibrate sensors

Article • 05/06/2023

Sensors in a device based on the magnetometer – the compass, inclinometer and orientation sensor - can require calibration due to environmental factors. The [MagnetometerAccuracy](#) enumeration can help determine when a device is in need of calibration.

Important APIs

- [Windows.Devices.Sensors](#)
- [Windows.Devices.Sensors.Custom](#)

When to calibrate the magnetometer

The [MagnetometerAccuracy](#) enumeration has four values that help you determine if the device your app is running on needs to be calibrated. If a device needs to be calibrated, you should let the user know that calibration is needed. However, you should not prompt the user to calibrate too frequently. We recommend no more than once every 10 minutes.

 Expand table

Value	Description
Unknown	The sensor driver could not report the current accuracy. This does not necessarily mean the device is out of calibration. It is up to your app to decide the best course of action if Unknown is returned. If your app is dependent on an accurate sensor reading, you may want to prompt the user to calibrate the device.
Unreliable	There is currently a high degree of inaccuracy in the magnetometer. Apps should always ask for a calibration from the user when this value is first returned.
Approximate	The data is accurate enough for some applications. A virtual reality app, that only needs to know if the user has moved the device up/down or left/right, can continue without calibration. Apps that need an absolute heading, like a navigation app that needs to know what direction you are driving in order to give you directions, need to ask for calibration.
High	The data is precise. No calibration is needed, even for apps that need to know an absolute heading such as augmented reality or navigation apps.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Sensor orientation

Article • 05/06/2023

Sensor data from the [Accelerometer](#), [Gyrometer](#), [Compass](#), [Inclinometer](#), and [OrientationSensor](#) classes is defined by their reference axes. These axes are defined by the device's reference frame and rotate with the device as the user turns it. If your app supports automatic rotation and reorients itself to accommodate the device as the user rotates it, you must adjust your sensor data for the rotation before using it.

Important APIs

- [Windows.Devices.Sensors](#)
- [Windows.Devices.Sensors.Custom](#)

Display orientation vs device orientation

In order to understand the reference axes for sensors, you need to distinguish display orientation from device orientation. Display orientation is the direction text and images are displayed on the screen whereas device orientation is the physical positioning of the device.

ⓘ Note

The positive z-axis extends out from the device screen, as show in the following

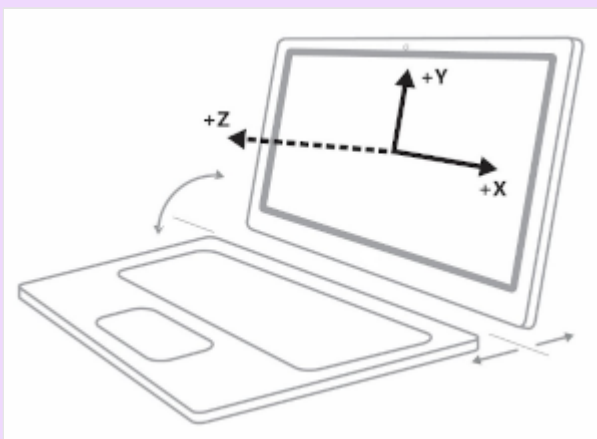
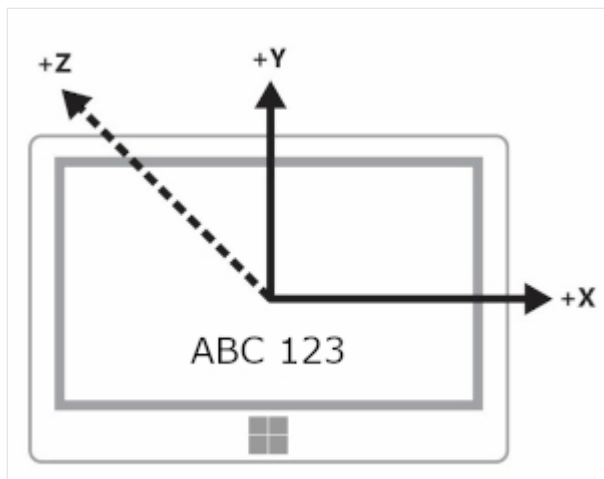


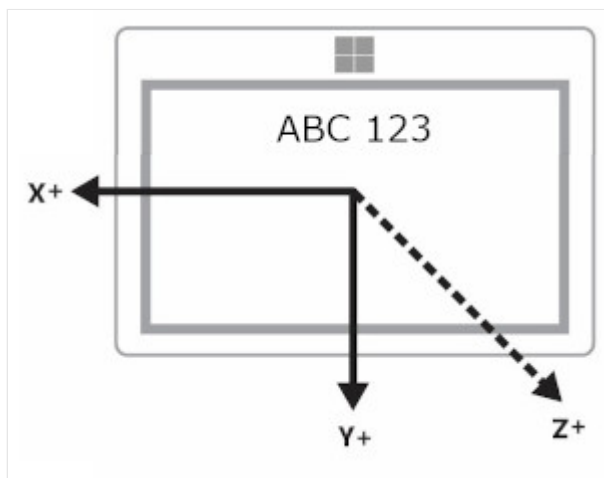
image.

In the following diagrams, both the device and display orientation are in [Landscape](#) (the sensor axes shown are specific to landscape orientation).

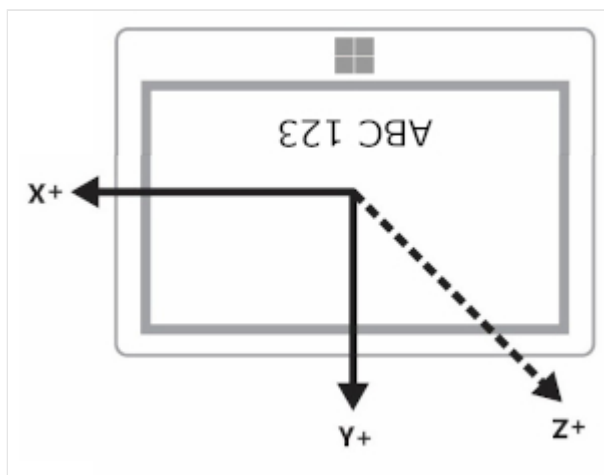
This diagram shows both the display and device orientation in [Landscape](#).



This next diagram shows both the display and device orientation in [LandscapeFlipped](#).



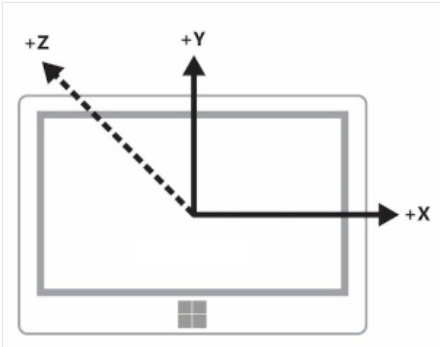
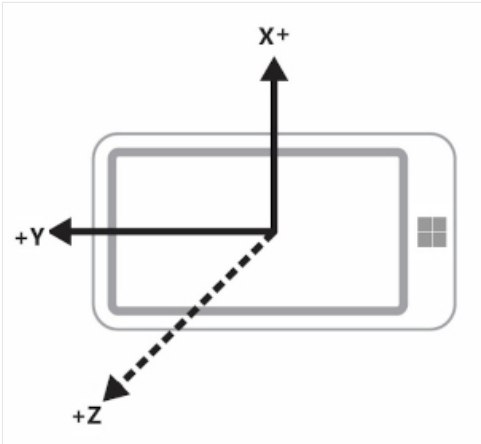
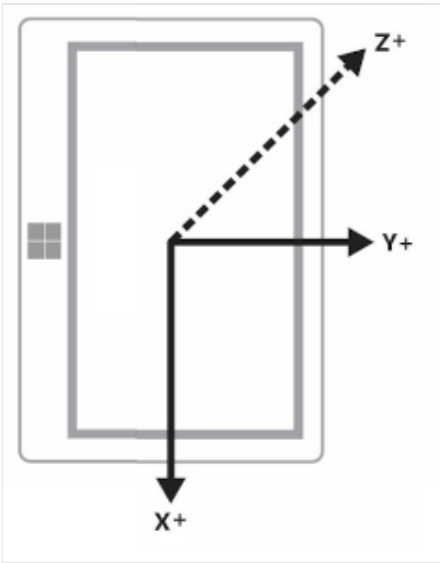
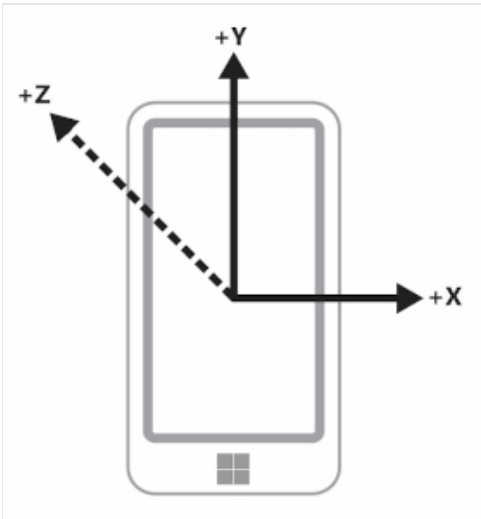
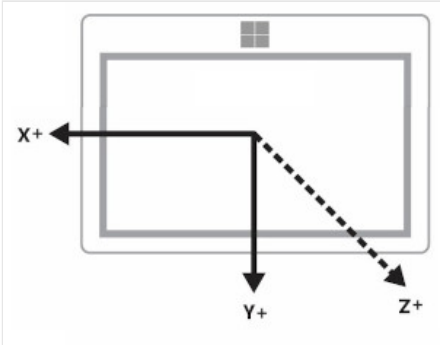
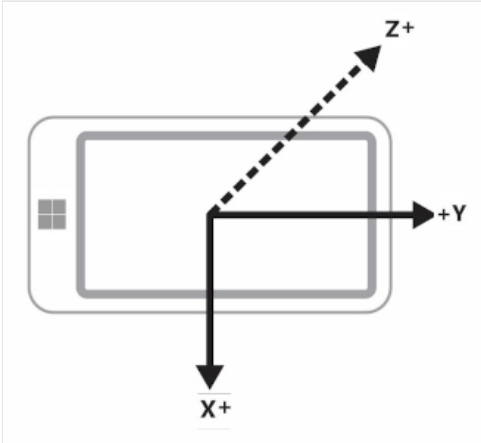
This final diagram shows the display orientation in Landscape while the device orientation is [LandscapeFlipped](#).

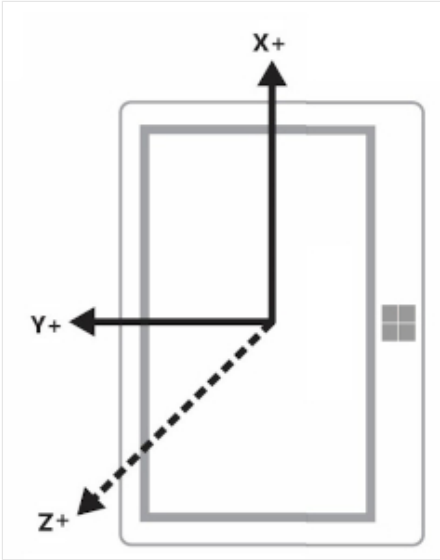
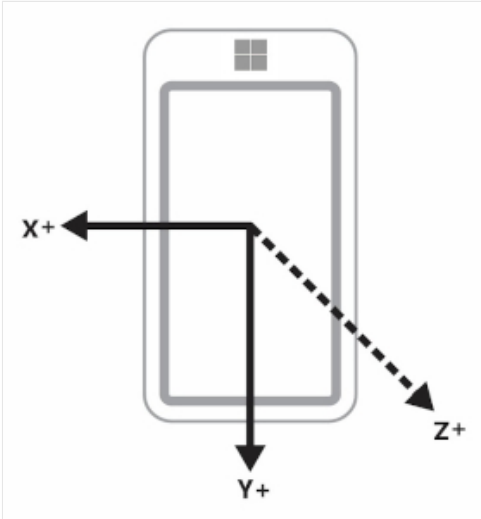


You can query the orientation values through the [DisplayInformation](#) class by using the [GetForCurrentView](#) method with the [CurrentOrientation](#) property. Then you can create logic by comparing against the [DisplayOrientations](#) enumeration. Remember that for every orientation you support, you have to support a conversion of the reference axes to that orientation.

Landscape-first vs portrait-first devices

Manufacturers produce both landscape-first and portrait-first devices. The reference frame varies between landscape-first devices (like desktops and laptops) and portrait-first devices (like phones and some tablets). The following table shows the sensor axes for both landscape-first and portrait-first devices.

Orientation	Landscape-first	Portrait-first
Landscape		
Portrait		
LandscapeFlipped		

Orientation	Landscape-first	Portrait-first
PortraitFlipped		

Devices broadcasting display and headless devices

Some devices have the ability to broadcast the display to another device. For example, you could take a tablet and broadcast the display to a projector that will be in landscape orientation. In this scenario, it is important to keep in mind that the device orientation is based on the original device, not the one presenting the display. So an accelerometer would report data for the tablet.

Furthermore, some devices do not have a display. With these devices, the default orientation for these devices is portrait.

Display orientation and compass heading

Compass heading depends upon the reference axes and so it changes with the device orientation. You compensate based on the this table (assume the user is facing north).

Display orientation	Reference axis for compass heading	API compass heading when facing north (landscape-first)	API compass heading when facing north (portrait-first)	Compass heading compensation (landscape-first)	Compass heading compensation (portrait-first)

Display orientation	Reference axis for compass heading	API compass heading when facing north (landscape-first)	API compass heading when facing north (portrait-first)	Compass heading compensation (landscape-first)	Compass heading compensation (portrait-first)
Landscape	-Z	0	270	Heading	(Heading + 90) % 360
Portrait	Y	90	0	(Heading + 270) % 360	Heading
LandscapeFlipped	Z	180	90	(Heading + 180) % 360	(Heading + 270) % 360
PortraitFlipped	Y	270	180	(Heading + 90) % 360	(Heading + 180) % 360

Modify the compass heading as shown in the table in order to correctly display the heading. The following code snippet demonstrates how to do this.

C#

```
private void ReadingChanged(object sender, CompassReadingChangedEventArgs e)
{
    double heading = e.Reading.HeadingMagneticNorth;
    double displayOffset;

    // Calculate the compass heading offset based on
    // the current display orientation.
    DisplayInformation displayInfo = DisplayInformation.GetForCurrentView();

    switch (displayInfo.CurrentOrientation)
    {
        case DisplayOrientations.Landscape:
            displayOffset = 0;
            break;
        case DisplayOrientations.Portrait:
            displayOffset = 270;
            break;
        case DisplayOrientations.LandscapeFlipped:
            displayOffset = 180;
            break;
        case DisplayOrientations.PortraitFlipped:
            displayOffset = 90;
            break;
    }

    double displayCompensatedHeading = (heading + displayOffset) % 360;
```

```
    // Update the UI...  
}
```

Display orientation with the accelerometer and gyrometer

This table converts accelerometer and gyrometer data for display orientation.

Reference axes	X	Y	Z
Landscape	X	Y	Z
Portrait	Y	-X	Z
LandscapeFlipped	-X	-Y	Z
PortraitFlipped	-Y	X	Z

The following code example applies these conversions to the gyrometer.

C#

```
private void ReadingChanged(object sender, GyrometerReadingChangedEventArgs e)  
{  
    double x_Axis;  
    double y_Axis;  
    double z_Axis;  
  
    GyrometerReading reading = e.Reading;  
  
    // Calculate the gyrometer axes based on  
    // the current display orientation.  
    DisplayInformation displayInfo = DisplayInformation.GetForCurrentView();  
    switch (displayInfo.CurrentOrientation)  
    {  
        case DisplayOrientations.Landscape:  
            x_Axis = reading.AngularVelocityX;  
            y_Axis = reading.AngularVelocityY;  
            z_Axis = reading.AngularVelocityZ;  
            break;  
        case DisplayOrientations.Portrait:  
            x_Axis = reading.AngularVelocityY;  
            y_Axis = -1 * reading.AngularVelocityX;  
            z_Axis = reading.AngularVelocityZ;  
            break;  
        case DisplayOrientations.LandscapeFlipped:  
            x_Axis = -1 * reading.AngularVelocityX;
```

```

        y_Axis = -1 * reading.AngularVelocityY;
        z_Axis = reading.AngularVelocityZ;
        break;
    case DisplayOrientations.PortraitFlipped:
        x_Axis = -1 * reading.AngularVelocityY;
        y_Axis = reading.AngularVelocityX;
        z_Axis = reading.AngularVelocityZ;
        break;
    }

    // Update the UI...
}

```

Display orientation and device orientation

The [OrientationSensor](#) data must be changed in a different way. Think of these different orientations as rotations counterclockwise to the z-axis, so we need to reverse the rotation to get back the user's orientation. For quaternion data, we can use Euler's formula to define a rotation with a reference quaternion, and we can also use a reference rotation matrix.

$$\mathbf{q} = e^{\frac{1}{2}\theta(u_x\mathbf{i}+u_y\mathbf{j}+u_z\mathbf{k})} = \cos\frac{1}{2}\theta + (u_x\mathbf{i} + u_y\mathbf{j} + u_z\mathbf{k}) \sin\frac{1}{2}\theta$$

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

To get the relative orientation you want, multiply the reference object against the absolute object. Note that this math is not commutative.

$$\mathbf{R}_{\text{rel}} = \mathbf{R}_{\text{ref}} \times \mathbf{R}_{\text{abs}}$$

$$\mathbf{q}_{\text{rel}} = \mathbf{q}_{\text{ref}} \times \mathbf{q}_{\text{abs}}$$

In the preceding expression, the absolute object is returned by the sensor data.

Display orientation	Counterclockwise rotation around Z	Reference quaternion (reverse rotation)	Reference rotation matrix (reverse rotation)
Landscape	0	$1 + 0\mathbf{i} + 0\mathbf{j} + 0\mathbf{k}$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Display orientation	Counterclockwise rotation around Z	Reference quaternion (reverse rotation)	Reference rotation matrix (reverse rotation)
Portrait	90	$\cos(-45^\circ) + (i + j + k) \sin(-45^\circ)$	$\begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
LandscapeFlipped	180	$0 - i - j - k$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
PortraitFlipped	270	$\cos(-135^\circ) + (i + j + k) \sin(-135^\circ)$	$\begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

See also

[Integrating Motion and Orientation Sensors](#)

Use the accelerometer

Article • 05/06/2023

Learn how to use the accelerometer to respond to user movement.

Important APIs

- [Windows.Devices.Sensors](#)
- [Accelerometer](#)

Prerequisites

You should be familiar with Extensible Application Markup Language (XAML), Microsoft Visual C#, and events.

The device or emulator that you're using must support an accelerometer.

Create a simple accelerometer app

A simple game app relies on a single sensor, the accelerometer, as an input device. These apps typically use only one or two axes for input; but they may also use the shake event as another input source.

ⓘ Note

For a more complete implementation, see the [accelerometer sample](#).

Instructions

- Create a new project, choosing a **Blank App (Universal Windows)** from the **Visual C#** project templates.
- Open your project's MainPage.xaml.cs file and replace the existing code with the following.

C#

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
```

```

using Windows.Foundation;
using Windows.Foundation.Collections;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Controls.Primitives;
using Windows.UI.Xaml.Data;
using Windows.UI.Xaml.Input;
using Windows.UI.Xaml.Media;
using Windows.UI.Xaml.Navigation;

// Required to support the core dispatcher and the accelerometer

using Windows.UI.Core;
using Windows.Devices.Sensors;

namespace App1
{
    public sealed partial class MainPage : Page
    {
        // Sensor and dispatcher variables
        private Accelerometer _accelerometer;

        // This event handler writes the current accelerometer reading
        to
        // the three acceleration text blocks on the app' s main page.

        private async void ReadingChanged(object sender,
AccelerometerReadingChangedEventArgs e)
        {
            await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, ()
=>
            {
                AccelerometerReading reading = e.Reading;
                txtXAxis.Text = String.Format("{0,5:0.00}",
reading.AccelerationX);
                txtYAxis.Text = String.Format("{0,5:0.00}",
reading.AccelerationY);
                txtZAxis.Text = String.Format("{0,5:0.00}",
reading.AccelerationZ);

            });
        }

        public MainPage()
        {
            this.InitializeComponent();
            _accelerometer = Accelerometer.GetDefault();

            if (_accelerometer != null)
            {
                // Establish the report interval
                uint minReportInterval =
_accelerometer.MinimumReportInterval;
                uint reportInterval = minReportInterval > 16 ?

```

```

minReportInterval : 16;
        _accelerometer.ReportInterval = reportInterval;

        // Assign an event handler for the reading-changed event
        _accelerometer.ReadingChanged += new
TypedEventHandler<Accelerometer, AccelerometerReadingChangedEventArgs>
(ReadingChanged);
    }
}
}
}

```

You'll need to rename the namespace in the previous snippet with the name you gave your project. For example, if you created a project named **AccelerometerCS**, you'd replace `namespace App1` with `namespace AccelerometerCS`.

- Open the file `MainPage.xaml` and replace the original contents with the following XML.

XML

```

<Page
x:Class="App1.MainPage"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="using:App1"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
mc:Ignorable="d">

    <Grid x:Name="LayoutRoot" Background="#FF0C0C0C">
        <TextBlock HorizontalAlignment="Left" Height="25"
Margin="8,20,0,0" TextWrapping="Wrap" Text="X-axis:" VerticalAlignment="Top"
Width="62" Foreground="#FFDE6E6"/>
        <TextBlock HorizontalAlignment="Left" Height="27"
Margin="8,49,0,0" TextWrapping="Wrap" Text="Y-axis:" VerticalAlignment="Top"
Width="62" Foreground="#FFF5F2F2"/>
        <TextBlock HorizontalAlignment="Left" Height="23"
Margin="8,80,0,0" TextWrapping="Wrap" Text="Z-axis:" VerticalAlignment="Top"
Width="62" Foreground="#FFF6F0F0"/>
        <TextBlock x:Name="txtXAxis" HorizontalAlignment="Left"
Height="15" Margin="70,16,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="61" Foreground="#FFF2F2F2"/>
        <TextBlock x:Name="txtYAxis" HorizontalAlignment="Left"
Height="15" Margin="70,49,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="53" Foreground="#FFF2EEEE"/>
        <TextBlock x:Name="txtZAxis" HorizontalAlignment="Left"
Height="15" Margin="70,80,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="53" Foreground="#FFFFFF8F8"/>
    </Grid>
</Page>

```

```
</Grid>
</Page>
```

You'll need to replace the first part of the class name in the previous snippet with the namespace of your app. For example, if you created a project named **AccelerometerCS**, you'd replace `x:Class="App1.MainPage"` with `x:Class="AccelerometerCS.MainPage"`. You should also replace `xmlns:local="using:App1"` with `xmlns:local="using:AccelerometerCS"`.

- Press F5 or select **Debug > Start Debugging** to build, deploy, and run the app.

Once the app is running, you can change the accelerometer values by moving the device or using the emulator tools.

- Stop the app by returning to Visual Studio and pressing Shift+F5 or select **Debug > Stop Debugging** to stop the app.

Explanation

The previous example demonstrates how little code you'll need to write in order to integrate accelerometer input in your app.

The app establishes a connection with the default accelerometer in the **MainPage** method.

C#

```
_accelerometer = Accelerometer.Default();
```

The app establishes the report interval within the **MainPage** method. This code retrieves the minimum interval supported by the device and compares it to a requested interval of 16 milliseconds (which approximates a 60-Hz refresh rate). If the minimum supported interval is greater than the requested interval, the code sets the value to the minimum. Otherwise, it sets the value to the requested interval.

C#

```
uint minReportInterval = _accelerometer.MinimumReportInterval;
uint reportInterval = minReportInterval > 16 ? minReportInterval : 16;
_accelerometer.ReportInterval = reportInterval;
```

The new accelerometer data is captured in the **ReadingChanged** method. Each time the sensor driver receives new data from the sensor, it passes the values to your app using

this event handler. The app registers this event handler on the following line.

C#

```
_accelerometer.ReadingChanged += new TypedEventHandler<Accelerometer, AccelerometerReadingChangedEventArgs>(ReadingChanged);
```

These new values are written to the TextBlocks found in the project's XAML.

XML

```
<TextBlock x:Name="txtXAxis" HorizontalAlignment="Left" Height="15"
Margin="70,16,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="61" Foreground="#FFF2F2F2"/>
  <TextBlock x:Name="txtYAxis" HorizontalAlignment="Left" Height="15"
Margin="70,49,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="53" Foreground="#FFF2EEEE"/>
  <TextBlock x:Name="txtZAxis" HorizontalAlignment="Left" Height="15"
Margin="70,80,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="53" Foreground="#FFFFFF8F8"/>
```

Use the compass

Article • 05/06/2023

Learn how to use the compass to determine the current heading.

Important APIs

- [Windows.Devices.Sensors](#)
- [Compass](#)

Prerequisites

You should be familiar with Extensible Application Markup Language (XAML), Microsoft Visual C#, and events.

The device or emulator that you're using must support a compass.

Create a simple compass app

An app can retrieve the current heading with respect to magnetic, or true, north. Navigation apps use the compass to determine the direction a device is facing and then orient the map accordingly.

ⓘ Note

For a more complete implementation, see the [compass sample](#) [↗].

Instructions

- Create a new project, choosing a **Blank App (Universal Windows)** from the **Visual C#** project templates.
- Open your project's MainPage.xaml.cs file and replace the existing code with the following.

C#

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
```

```

using Windows.Foundation;
using Windows.Foundation.Collections;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Controls.Primitives;
using Windows.UI.Xaml.Data;
using Windows.UI.Xaml.Input;
using Windows.UI.Xaml.Media;
using Windows.UI.Xaml.Navigation;

using Windows.UI.Core; // Required to access the core dispatcher object
using Windows.Devices.Sensors; // Required to access the sensor platform
and the compass

namespace App1
{
    /// <summary>
    /// An empty page that can be used on its own or navigated to within
a Frame.
    /// </summary>
    public sealed partial class MainPage : Page
    {
        private Compass _compass; // Our app' s compass object

        // This event handler writes the current compass reading to
        // the textblocks on the app' s main page.

        private async void ReadingChanged(object sender,
CompassReadingChangedEventArgs e)
        {
            =>
            await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, ()
            {
                CompassReading reading = e.Reading;
                txtMagnetic.Text = String.Format("{0,5:0.00}",
reading.HeadingMagneticNorth);
                if (reading.HeadingTrueNorth.HasValue)
                    txtNorth.Text = String.Format("{0,5:0.00}",
reading.HeadingTrueNorth);
                else
                    txtNorth.Text = "No reading.";
            });
        }

        public MainPage()
        {
            this.InitializeComponent();
            _compass = Compass.Default; // Get the default compass
object

            // Assign an event handler for the compass reading-changed
event

            if (_compass != null)
            {

```

```

        // Establish the report interval for all scenarios
        uint minReportInterval = _compass.MinimumReportInterval;
        uint reportInterval = minReportInterval > 16 ?
minReportInterval : 16;
        _compass.ReportInterval = reportInterval;
        _compass.ReadingChanged += new
TypedEventHandler<Compass, CompassReadingChangedEventArgs>(ReadingChanged);
    }
}
}
}

```

You'll need to rename the namespace in the previous snippet with the name you gave your project. For example, if you created a project named **CompassCS**, you'd replace `namespace App1` with `namespace CompassCS`.

- Open the file `MainPage.xaml` and replace the original contents with the following XML.

XML

```

<Page
    x:Class="App1.MainPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:App1"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
    mc:Ignorable="d">

    <Grid x:Name="LayoutRoot" Background="#FF0C0C0C">
        <TextBlock HorizontalAlignment="Left" Height="22"
Margin="8,18,0,0" TextWrapping="Wrap" Text="Magnetic Heading:"
VerticalAlignment="Top" Width="104" Foreground="#FFFBF9F9"/>
        <TextBlock HorizontalAlignment="Left" Height="18"
Margin="8,58,0,0" TextWrapping="Wrap" Text="True North Heading:"
VerticalAlignment="Top" Width="104" Foreground="#FFF3F3F3"/>
        <TextBlock x:Name="txtMagnetic" HorizontalAlignment="Left"
Height="22" Margin="130,18,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="116" Foreground="#FFFBF6F6"/>
        <TextBlock x:Name="txtNorth" HorizontalAlignment="Left"
Height="18" Margin="130,58,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="116" Foreground="#FFF5F1F1"/>

    </Grid>
</Page>

```

You'll need to replace the first part of the class name in the previous snippet with the namespace of your app. For example, if you created a project named **CompassCS**, you'd

replace `x:Class="App1.MainPage"` with `x:Class="CompassCS.MainPage"`. You should also replace `xmlns:local="using:App1"` with `xmlns:local="using:CompassCS"`.

- Press F5 or select **Debug > Start Debugging** to build, deploy, and run the app.

Once the app is running, you can change the compass values by moving the device or using the emulator tools.

- Stop the app by returning to Visual Studio and pressing Shift+F5 or select **Debug > Stop Debugging** to stop the app.

Explanation

The previous example demonstrates how little code you'll need to write in order to integrate compass input in your app.

The app establishes a connection with the default compass in the **MainPage** method.

C#

```
_compass = Compass.Default(); // Get the default compass object
```

The app establishes the report interval within the **MainPage** method. This code retrieves the minimum interval supported by the device and compares it to a requested interval of 16 milliseconds (which approximates a 60-Hz refresh rate). If the minimum supported interval is greater than the requested interval, the code sets the value to the minimum. Otherwise, it sets the value to the requested interval.

C#

```
uint minReportInterval = _compass.MinimumReportInterval;  
uint reportInterval = minReportInterval > 16 ? minReportInterval : 16;  
_compass.ReportInterval = reportInterval;
```

The new compass data is captured in the **ReadingChanged** method. Each time the sensor driver receives new data from the sensor, it passes the values to your app using this event handler. The app registers this event handler on the following line.

C#

```
_compass.ReadingChanged += new TypedEventHandler<Compass,  
CompassReadingChangedEventArgs>(ReadingChanged);
```

These new values are written to the TextBlocks found in the project's XAML.

XML

```
<TextBlock HorizontalAlignment="Left" Height="22" Margin="8,18,0,0"
TextWrapping="Wrap" Text="Magnetic Heading:" VerticalAlignment="Top"
Width="104" Foreground="#FFFBF9F9"/>
<TextBlock HorizontalAlignment="Left" Height="18" Margin="8,58,0,0"
TextWrapping="Wrap" Text="True North Heading:" VerticalAlignment="Top"
Width="104" Foreground="#FFF3F3F3"/>
<TextBlock x:Name="txtMagnetic" HorizontalAlignment="Left" Height="22"
Margin="130,18,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="116" Foreground="#FFFBF6F6"/>
<TextBlock x:Name="txtNorth" HorizontalAlignment="Left" Height="18"
Margin="130,58,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="116" Foreground="#FFF5F1F1"/>
```

Use the gyrometer

Article • 05/06/2023

Learn how to use the gyrometer to detect changes in user movement.

Important APIs

- [Windows.Devices.Sensors](#)
- [Gyrometer](#)

Prerequisites

You should be familiar with Extensible Application Markup Language (XAML), Microsoft Visual C#, and events.

The device or emulator that you're using must support a gyrometer.

Create a simple gyrometer app

Gyrometers compliment accelerometers as game controllers. The accelerometer can measure linear motion while the gyrometer measures angular velocity or rotational motion.

ⓘ Note

For a more complete implementation, see the [gyrometer sample](#) [↗].

Instructions

- Create a new project, choosing a **Blank App (Universal Windows)** from the **Visual C#** project templates.
- Open your project's MainPage.xaml.cs file and replace the existing code with the following.

C#

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
```

```

using Windows.Foundation;
using Windows.Foundation.Collections;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Controls.Primitives;
using Windows.UI.Xaml.Data;
using Windows.UI.Xaml.Input;
using Windows.UI.Xaml.Media;
using Windows.UI.Xaml.Navigation;

using Windows.UI.Core; // Required to access the core dispatcher object
using Windows.Devices.Sensors; // Required to access the sensor platform
and the gyrometer

namespace App1
{
    /// <summary>
    /// An empty page that can be used on its own or navigated to within
a Frame.
    /// </summary>
    public sealed partial class MainPage : Page
    {
        private Gyrometer _gyrometer; // Our app' s gyrometer object

        // This event handler writes the current gyrometer reading to
        // the three textblocks on the app' s main page.

        private async void ReadingChanged(object sender,
GyrometerReadingChangedEventArgs e)
        {
            await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, ()
=>
            {
                GyrometerReading reading = e.Reading;
                txtXAxis.Text = String.Format("{0,5:0.00}",
reading.AngularVelocityX);
                txtYAxis.Text = String.Format("{0,5:0.00}",
reading.AngularVelocityY);
                txtZAxis.Text = String.Format("{0,5:0.00}",
reading.AngularVelocityZ);
            });
        }

        public MainPage()
        {
            this.InitializeComponent();
            _gyrometer = Gyrometer.Default(); // Get the default
gyrometer sensor object

            if (_gyrometer != null)
            {
                // Establish the report interval for all scenarios
                uint minReportInterval =
_gyrometer.MinimumReportInterval;

```

```

        uint reportInterval = minReportInterval > 16 ?
minReportInterval : 16;
        _gyrometer.ReportInterval = reportInterval;

        // Assign an event handler for the gyrometer reading-
changed event
        _gyrometer.ReadingChanged += new
TypedEventHandler<Gyrometer, GyrometerReadingChangedEventArgs>
(ReadingChanged);
    }

}
}
}

```

You'll need to rename the namespace in the previous snippet with the name you gave your project. For example, if you created a project named **GyrometerCS**, you'd replace `namespace App1` with `namespace GyrometerCS`.

- Open the file `MainPage.xaml` and replace the original contents with the following XML.

XML

```

<Page
x:Class="App1.MainPage"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="using:App1"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
mc:Ignorable="d">

    <Grid x:Name="LayoutRoot" Background="#FF0C0C0C">
        <TextBlock HorizontalAlignment="Left" Height="23"
Margin="8,8,0,0" TextWrapping="Wrap" Text="X-Axis:" VerticalAlignment="Top"
Width="46" Foreground="#FFFDFFFD"/>
        <TextBlock x:Name="txtXAxis" HorizontalAlignment="Left"
Height="23" Margin="67,8,0,0" TextWrapping="Wrap" VerticalAlignment="Top"
Width="88" Foreground="#FFFDFAFA"/>
        <TextBlock HorizontalAlignment="Left" Height="20"
Margin="8,52,0,0" TextWrapping="Wrap" Text="Y Axis:" VerticalAlignment="Top"
Width="46" Foreground="White"/>
        <TextBlock x:Name="txtYAxis" HorizontalAlignment="Left"
Height="24" Margin="54,48,0,0" TextWrapping="Wrap" VerticalAlignment="Top"
Width="80" Foreground="#FFFBFBFB"/>
        <TextBlock HorizontalAlignment="Left" Height="21"
Margin="8,93,0,0" TextWrapping="Wrap" Text="Z Axis:" VerticalAlignment="Top"
Width="46" Foreground="#FFFEFBFB"/>
        <TextBlock x:Name="txtZAxis" HorizontalAlignment="Left"
Height="21" Margin="54,93,0,0" TextWrapping="Wrap" VerticalAlignment="Top"

```

```
Width="63" Foreground="#FFF8F3F3"/>
```

```
</Grid>  
</Page>
```

You'll need to replace the first part of the class name in the previous snippet with the namespace of your app. For example, if you created a project named **GyrometerCS**, you'd replace `x:Class="App1.MainPage"` with `x:Class="GyrometerCS.MainPage"`. You should also replace `xmlns:local="using:App1"` with `xmlns:local="using:GyrometerCS"`.

- Press F5 or select **Debug > Start Debugging** to build, deploy, and run the app.

Once the app is running, you can change the gyrometer values by moving the device or using the emulator tools.

- Stop the app by returning to Visual Studio and pressing Shift+F5 or select **Debug > Stop Debugging** to stop the app.

Explanation

The previous example demonstrates how little code you'll need to write in order to integrate gyrometer input in your app.

The app establishes a connection with the default gyrometer in the **MainPage** method.

C#

```
_gyrometer = Gyrometer.GetDefault(); // Get the default gyrometer sensor  
object
```

The app establishes the report interval within the **MainPage** method. This code retrieves the minimum interval supported by the device and compares it to a requested interval of 16 milliseconds (which approximates a 60-Hz refresh rate). If the minimum supported interval is greater than the requested interval, the code sets the value to the minimum. Otherwise, it sets the value to the requested interval.

C#

```
uint minReportInterval = _gyrometer.MinimumReportInterval;  
uint reportInterval = minReportInterval > 16 ? minReportInterval : 16;  
_gyrometer.ReportInterval = reportInterval;
```

The new gyrometer data is captured in the **ReadingChanged** method. Each time the sensor driver receives new data from the sensor, it passes the values to your app using

this event handler. The app registers this event handler on the following line.

C#

```
_gyrometer.ReadingChanged += new TypedEventHandler<Gyrometer,  
GyrometerReadingChangedEventArgs>(ReadingChanged);
```

These new values are written to the TextBlocks found in the project's XAML.

XML

```
<TextBlock HorizontalAlignment="Left" Height="23" Margin="8,8,0,0"  
TextWrapping="Wrap" Text="X-Axis:" VerticalAlignment="Top" Width="46"  
Foreground="#FFDFDFD" />  
<TextBlock x:Name="txtXAxis" HorizontalAlignment="Left" Height="23"  
Margin="67,8,0,0" TextWrapping="Wrap" VerticalAlignment="Top" Width="88"  
Foreground="#FFDFAFA" />  
<TextBlock HorizontalAlignment="Left" Height="20" Margin="8,52,0,0"  
TextWrapping="Wrap" Text="Y Axis:" VerticalAlignment="Top" Width="46"  
Foreground="White" />  
<TextBlock x:Name="txtYAxis" HorizontalAlignment="Left" Height="24"  
Margin="54,48,0,0" TextWrapping="Wrap" VerticalAlignment="Top" Width="80"  
Foreground="#FFBFBFB" />  
<TextBlock HorizontalAlignment="Left" Height="21" Margin="8,93,0,0"  
TextWrapping="Wrap" Text="Z Axis:" VerticalAlignment="Top" Width="46"  
Foreground="#FFEFBFB" />  
<TextBlock x:Name="txtZAxis" HorizontalAlignment="Left" Height="21"  
Margin="54,93,0,0" TextWrapping="Wrap" VerticalAlignment="Top" Width="63"  
Foreground="#FF8F3F3" />
```

Related topics

- [Gyrometer Sample](#) ↗

Use the inclinometer

Article • 05/06/2023

Learn how to use the inclinometer to determine pitch, roll, and yaw.

Important APIs

- [Windows.Devices.Sensors](#)
- [Inclinometer](#)

Prerequisites

You should be familiar with Extensible Application Markup Language (XAML), Microsoft Visual C#, and events.

The device or emulator that you're using must support an inclinometer.

Create a simple inclinometer app

Some 3-D games require an inclinometer as an input device. One common example is the flight simulator, which maps the three axes of the inclinometer (X, Y, and Z) to the elevator, aileron, and rudder inputs of the aircraft.

ⓘ Note

For a more complete implementation, see the [inclinometer sample](#) [↗].

Instructions

- Create a new project, choosing a **Blank App (Universal Windows)** from the **Visual C#** project templates.
- Open your project's MainPage.xaml.cs file and replace the existing code with the following.

C#

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
```

```

using Windows.Foundation;
using Windows.Foundation.Collections;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Controls.Primitives;
using Windows.UI.Xaml.Data;
using Windows.UI.Xaml.Input;
using Windows.UI.Xaml.Media;
using Windows.UI.Xaml.Navigation;

using Windows.UI.Core;
using Windows.Devices.Sensors;

namespace App1
{
    /// <summary>
    /// An empty page that can be used on its own or navigated to within
a Frame.
    /// </summary>
    public sealed partial class MainPage : Page
    {
        private Inclinometer _inclinometer;

        // This event handler writes the current inclinometer reading to
        // the three text blocks on the app' s main page.

        private async void ReadingChanged(object sender,
InclinometerReadingChangedEventArgs e)
        {
            => await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, ()
            {
                InclinometerReading reading = e.Reading;
                txtPitch.Text = String.Format("{0,5:0.00}",
reading.PitchDegrees);
                txtRoll.Text = String.Format("{0,5:0.00}",
reading.RollDegrees);
                txtYaw.Text = String.Format("{0,5:0.00}",
reading.YawDegrees);
            });
        }

        public MainPage()
        {
            this.InitializeComponent();
            _inclinometer = Inclinometer.GetDefault();

            if (_inclinometer != null)
            {
                // Establish the report interval for all scenarios
                uint minReportInterval =
_inclinometer.MinimumReportInterval;
                uint reportInterval = minReportInterval > 16 ?

```

```

minReportInterval : 16;
        _inclinometer.ReportInterval = reportInterval;

        // Establish the event handler
        _inclinometer.ReadingChanged += new
TypedEventHandler<Inclinometer, InclinometerReadingChangedEventArgs>
(ReadingChanged);
    }
}
}
}
}

```

You'll need to rename the namespace in the previous snippet with the name you gave your project. For example, if you created a project named **InclinometerCS**, you'd replace `namespace App1` with `namespace InclinometerCS`.

- Open the file `MainPage.xaml` and replace the original contents with the following XML.

XML

```

<Page
x:Class="App1.MainPage"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="using:App1"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
mc:Ignorable="d">

    <Grid x:Name="LayoutRoot" Background="#FF0C0C0C">
        <TextBlock HorizontalAlignment="Left" Height="21"
Margin="0,8,0,0" TextWrapping="Wrap" Text="Pitch: " VerticalAlignment="Top"
Width="45" Foreground="#FFF9F4F4"/>
        <TextBlock x:Name="txtPitch" HorizontalAlignment="Left"
Height="21" Margin="59,8,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="71" Foreground="#FFDF9F9"/>
        <TextBlock HorizontalAlignment="Left" Height="23"
Margin="0,29,0,0" TextWrapping="Wrap" Text="Roll:" VerticalAlignment="Top"
Width="55" Foreground="#FFF7F1F1"/>
        <TextBlock x:Name="txtRoll" HorizontalAlignment="Left"
Height="23" Margin="59,29,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="50" Foreground="#FFCF9F9"/>
        <TextBlock HorizontalAlignment="Left" Height="19"
Margin="0,56,0,0" TextWrapping="Wrap" Text="Yaw:" VerticalAlignment="Top"
Width="55" Foreground="#FFF7F3F3"/>
        <TextBlock x:Name="txtYaw" HorizontalAlignment="Left"
Height="19" Margin="55,56,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="54" Foreground="#FFF6F2F2"/>
    </Grid>
</Page>

```

```
</Grid>
</Page>
```

You'll need to replace the first part of the class name in the previous snippet with the namespace of your app. For example, if you created a project named **InclinometerCS**, you'd replace `x:Class="App1.MainPage"` with `x:Class="InclinometerCS.MainPage"`. You should also replace `xmlns:local="using:App1"` with `xmlns:local="using:InclinometerCS"`.

- Press F5 or select **Debug > Start Debugging** to build, deploy, and run the app.

Once the app is running, you can change the inclinometer values by moving the device or using the emulator tools.

- Stop the app by returning to Visual Studio and pressing Shift+F5 or select **Debug > Stop Debugging** to stop the app.

Explanation

The previous example demonstrates how little code you'll need to write in order to integrate inclinometer input in your app.

The app establishes a connection with the default inclinometer in the **MainPage** method.

C#

```
_inclinometer = Inclinometer.GetDefault();
```

The app establishes the report interval within the **MainPage** method. This code retrieves the minimum interval supported by the device and compares it to a requested interval of 16 milliseconds (which approximates a 60-Hz refresh rate). If the minimum supported interval is greater than the requested interval, the code sets the value to the minimum. Otherwise, it sets the value to the requested interval.

C#

```
uint minReportInterval = _inclinometer.MinimumReportInterval;
uint reportInterval = minReportInterval > 16 ? minReportInterval : 16;
_inclinometer.ReportInterval = reportInterval;
```

The new inclinometer data is captured in the **ReadingChanged** method. Each time the sensor driver receives new data from the sensor, it passes the values to your app using

this event handler. The app registers this event handler on the following line.

C#

```
_inclinometer.ReadingChanged += new TypedEventHandler<Inclinometer,
InclinometerReadingChangedEventArgs>(ReadingChanged);
```

These new values are written to the TextBlocks found in the project's XAML.

XML

```
<TextBlock HorizontalAlignment="Left" Height="21" Margin="0,8,0,0"
TextWrapping="Wrap" Text="Pitch: " VerticalAlignment="Top" Width="45"
Foreground="#FFF9F4F4"/>
<TextBlock x:Name="txtPitch" HorizontalAlignment="Left" Height="21"
Margin="59,8,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="71" Foreground="#FFFD9F9F"/>
<TextBlock HorizontalAlignment="Left" Height="23" Margin="0,29,0,0"
TextWrapping="Wrap" Text="Roll:" VerticalAlignment="Top" Width="55"
Foreground="#FFF7F1F1"/>
<TextBlock x:Name="txtRoll" HorizontalAlignment="Left" Height="23"
Margin="59,29,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="50" Foreground="#FFFCF9F9"/>
<TextBlock HorizontalAlignment="Left" Height="19" Margin="0,56,0,0"
TextWrapping="Wrap" Text="Yaw:" VerticalAlignment="Top" Width="55"
Foreground="#FFF7F3F3"/>
<TextBlock x:Name="txtYaw" HorizontalAlignment="Left" Height="19"
Margin="55,56,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="54" Foreground="#FFF6F2F2"/>
```

Feedback

Was this page helpful?

 Yes

 No

Provide product feedback [↗](#) | [Get help at Microsoft Q&A](#)